

NebuCore SoC

RISC-V-Based System-on-Chip for Volume-to-Time Dosage Control and Safety-Interlocked Nebulizer Actuation

1. Introduction

Nebulizer therapy requires a patient to inhale a prescribed liquid medication dose over a specific, clinically-determined duration. In most low-cost nebulizers this duration is left to the caregiver to estimate and time manually, which leads to inconsistent dosing, over- or under-treatment, and no safeguard against the device being left running after the medication is exhausted or the timed session has expired.

This project designs and verifies a RISC-V-based System-on-Chip that automates and hardware-secures this process end to end. The SoC accepts the liquid volume actually loaded into the device, converts that volume into an exact treatment time, drives the nebulizing element only for that duration, and enforces a hard safety lock so the unit cannot be silently re-enabled once the dose is complete.

What the project does

- Reads a caregiver-entered liquid volume (mL) through a GPIO-connected keypad/switch interface.
- Converts that volume into a treatment duration in firmware, using a fixed mL-to-time lookup, and loads the result into a memory-mapped Timer peripheral.
- Counts down the dose once per second, updating a Seven-Segment Display Controller with the time remaining and, at completion, a 'donE' message.
- Drives a piezoelectric nebulizing element through a PWM Generator for exactly the computed duration, with the enable line refreshed once per Timer tick so a core stall cannot leave the actuator running.
- Hands off the dose-complete condition to an independent Watchdog Timer IP, which latches a locked state, force-disables the PWM output, and drives an alarm — regardless of firmware activity — until an explicit system reset.

Why it matters

A discrete microcontroller and analog timer circuit can approximate this behaviour, but a purpose-built RISC-V SoC offers a single, verifiable platform: the processor sequences volume entry, dosage-time computation, and completion handling in firmware, while memory-mapped peripheral IPs handle the real-time actuation, display, and safety-interlock functions that firmware alone cannot guarantee with hard timing and glitch-free behaviour. The result is a compact, memory-mapped SoC suitable for a battery-powered handheld or tabletop medical-adjacent device, in which the safety interlock is physically decoupled from firmware — a firmware fault or race condition cannot, by itself, leave the actuator enabled.

The following sections present the top-level block diagram, the on-chip and off-chip signal lists, and the control/data path breakdown for the design.

2. Block Diagram

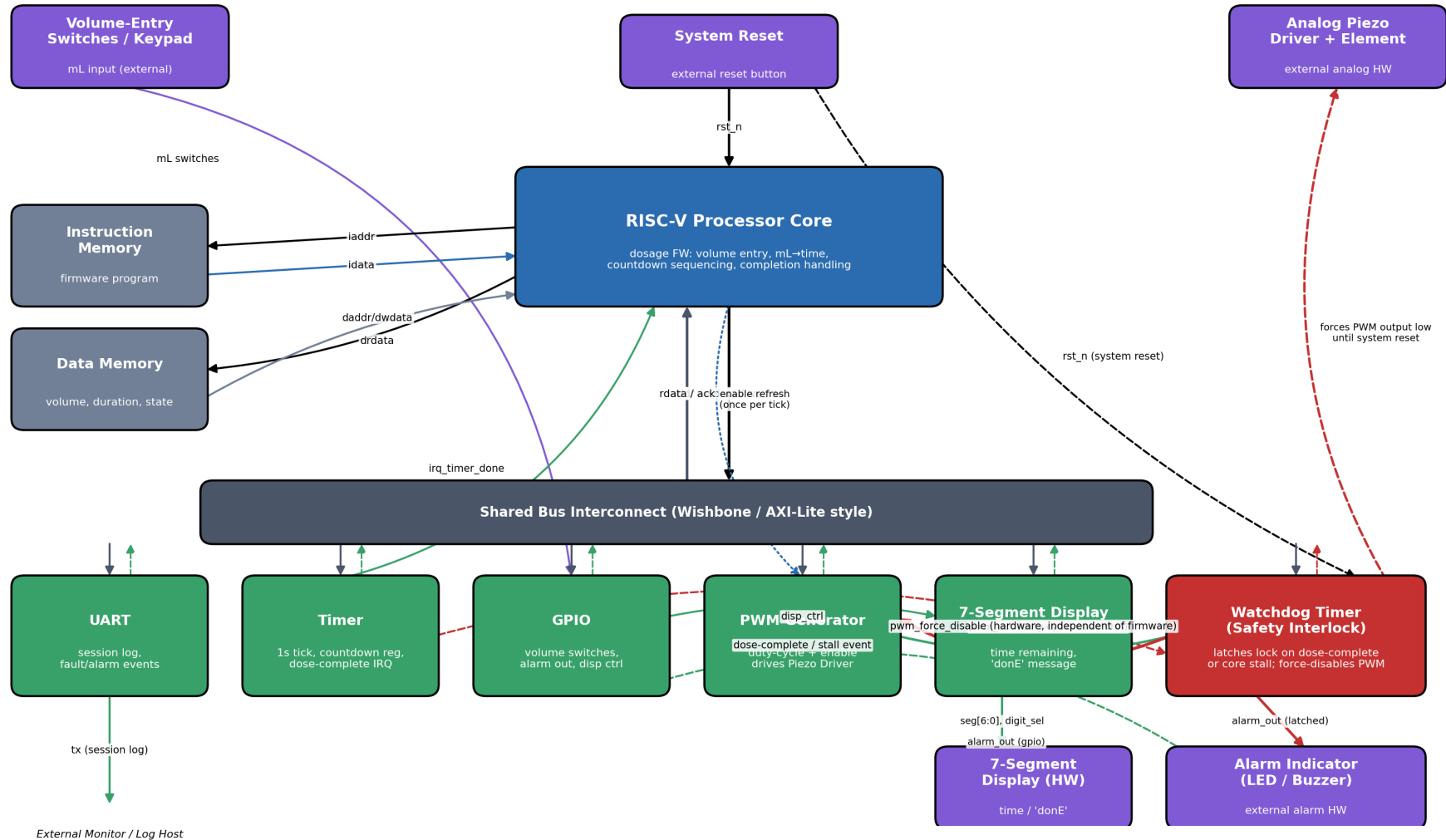
The figure on the following page shows the NebuCore SoC top-level block diagram: the RISC-V processor core, shared bus interconnect, instruction/data memory, and the six peripheral IPs (UART, Timer, GPIO, PWM Generator, Seven-Segment Display Controller, and Watchdog Timer), together with the off-chip devices they connect to (volume-entry switches, the piezo driver, the physical display, and the alarm indicator).

NebuCore SoC

RISC-V-Based System-on-Chip for Volume-to-Time Dosage Control and Safety-Interlocked Nebulizer Actuation — Block Diagram & Signal List

NebuCore SoC — Top-Level Block Diagram

RISC-V-Based SoC for Volume-to-Time Dosage Control and Safety-Interlocked Nebulizer Actuation



2.1 Internal Blocks of the Architecture

The SoC is organized around the provided RISC-V processor core as the central control unit, connected via a shared memory-mapped bus interconnect to instruction memory, data memory, and six peripheral IP blocks. Each internal block is summarized below; full register-level signals are given in the Signal List.

RISC-V Processor Core

Central control unit. Runs the dosage-control firmware: reads the caregiver-entered volume, performs the mL-to-time conversion, sequences the countdown, refreshes the PWM enable once per tick, updates the display, and hands off completion to the Watchdog.

Bus Interconnect (Wishbone / AXI-Lite style)

Shared memory-mapped fabric connecting the core to instruction memory, data memory, and every peripheral. Decodes the core's address into per-peripheral chip-selects and muxes read-data/acknowledge back to the core.

Instruction Memory

Stores the firmware program executed by the RISC-V core.

Data Memory

Stores the entered volume, the computed treatment duration, and session state used across the dosage-control routine.

UART

Logs session start, completion, and any fault/alarm events to an external monitor or log host over a serial link.

Timer

Generates the 1-second countdown tick from the loaded duration and raises the dose-complete interrupt at zero, driving both the display refresh and the Watchdog trigger.

GPIO

Reads the volume-entry switches/keypad and drives the display control lines and the pre-latch alarm output.

PWM Generator

Produces the variable duty-cycle waveform that drives the external analog Piezo Driver. Enable must be refreshed by firmware once per tick, and can be force-disabled directly by the Watchdog Timer regardless of firmware state.

Seven-Segment Display Controller

Memory-mapped digit driver that shows time remaining during the countdown and the 'donE' completion message, updated by firmware once per Timer tick.

Watchdog Timer (Safety Interlock)

Independent hardware safety block. Latches a locked state on dose completion or on core-stall detection, force-disables the PWM Generator's output, and drives the alarm signal until the system is explicitly reset — the single block guaranteeing the actuator cannot be silently re-enabled.

3. Signal List

The tables on the following pages list every signal in the NebuCore SoC, grouped by block: the RISC-V core's top-level ports, instruction and data memory, the shared bus interconnect, and each of the six peripheral IPs, followed by the external/off-chip signals. Direction is given from the perspective of the named block.

Signal List

Signals are grouped by block. Direction is given from the perspective of the named block. "Bus" indicates the signal is carried as part of the shared memory-mapped bus interconnect (address, write-data, read-data, write-enable, chip-select, acknowledge) between the RISC-V core and every peripheral; only bus signals with block-specific meaning are broken out separately below.

1. RISC-V Processor Core (top-level control ports)			
Signal	Width	Dir.	Description
clk	1	in	System clock.
rst_n	1	in	Active-low system reset, sourced from the external Reset button; also broadcast to the Watchdog Timer so a manual reset can clear a latched lock.
bus_addr	32	out	Address driven onto the shared bus for the current memory or peripheral access.
bus_wdata	32	out	Write-data driven onto the shared bus (register writes to peripherals, data-memory stores).
bus_rdata	32	in	Read-data returned from the addressed memory or peripheral.
bus_we	1	out	Bus write-enable; qualifies bus_wdata as a valid write.
bus_sel	1	out	Bus byte/word select strobe for the current transaction.
bus_ack	1	in	Bus acknowledge / ready from the addressed slave, used to extend transactions where needed.
irq_timer_done	1	in	Interrupt from the Timer peripheral: fires each 1-second tick and again at dose-complete (count = 0).
irq_wdt	1	in	Optional status interrupt from the Watchdog Timer (e.g., core-stall detection), used for logging; the lock itself is enforced in hardware independent of this line.

2. Instruction Memory			
Signal	Width	Dir.	Description
iaddr	32	in	Instruction fetch address driven by the RISC-V core.
idata	32	out	Instruction word returned to the core's fetch stage.
ien	1	in	Fetch enable / chip-select from the core.

3. Data Memory			
Signal	Width	Dir.	Description
daddr	32	in	Data address for load/store of entered volume, computed duration, and session state.
dwwdata	32	in	Write data for stores from the core.
drdata	32	out	Read data returned to the core for loads.

dwe	1	in	Data memory write-enable.
dre	1	in	Data memory read-enable.

4. Shared Bus Interconnect

Signal	Width	Dir.	Description
m_addr / m_wdata / m_we	32/32/1	in	Master-side (core) address, write-data, and write-enable presented to the interconnect.
m_rdata / m_ack	32/1	out	Read-data and acknowledge returned to the core.
s_sel[N-1:0]	N	out	Per-peripheral chip-select/decode lines (one per attached IP: UART, Timer, GPIO, PWM Generator, 7-Segment Display Controller, Watchdog Timer).
s_addr / s_wdata / s_we	32/32/1	out	Address, write-data, and write-enable forwarded to the selected peripheral's register interface.
s_rdata / s_ack	32/1	in	Read-data and acknowledge muxed back from whichever peripheral is currently selected.

5. UART

Signal	Width	Dir.	Description
tx	1	out	Serial transmit line to the external monitor/log host; carries session start, completion, and fault/alarm log messages.
rx	1	in	Serial receive line (reserved for future host commands / diagnostics).
tx_data[7:0]	8	in	Byte to transmit, written by firmware via the bus.
tx_valid / tx_ready	1/1	in/out	Handshake: firmware asserts tx_valid to load a byte; peripheral asserts tx_ready when the transmit buffer is free.
rx_data[7:0] / rx_valid	8/1	out	Received byte and its valid strobe, readable by firmware via the bus.

6. Timer

Signal	Width	Dir.	Description
timer_load[15:0]	16	in	Countdown value (in seconds) loaded by firmware after the mL-to-time conversion.
timer_en	1	in	Countdown enable, set by firmware to start/stop the session timer.
timer_count[15:0]	16	out	Live countdown value, read by firmware once per tick to refresh the display.
tick_1s	1	internal	Internal 1 Hz strobe that decrements timer_count (not bus-visible; listed for clarity of timing behavior).
irq_done	1	out	Asserted when timer_count reaches zero — the dose-complete event that triggers PWM disable and the Watchdog lock.

7. GPIO

Signal	Width	Dir.	Description
gpio_in[7:0]	8	in	Volume-entry switch / keypad inputs (mL value), sampled by firmware.
gpio_out[7:0]	8	out	General-purpose outputs: display control strobes and the (pre-latch) alarm indicator line.
gpio_dir[7:0]	8	in	Per-pin direction configuration register.
gpio_irq	1	out	Optional interrupt-on-change for the volume-entry inputs, used instead of polling.

8. PWM Generator

Signal	Width	Dir.	Description
duty_cycle[7:0]	8	in	Duty-cycle register, set by firmware to control nebulizing intensity.
pwm_en	1	in	Enable bit; firmware must keep re-asserting this once per Timer tick, so a core stall lets the enable lapse rather than leaving the actuator running.
pwm_out	1	out	PWM drive waveform sent to the external analog Piezo Driver.
pwm_force_disable	1	in	Hardware override input driven directly by the Watchdog Timer; when asserted, forces pwm_out low regardless of pwm_en or firmware state.

9. Seven-Segment Display Controller

Signal	Width	Dir.	Description
digit_data[15:0]	16	in	Value to display (countdown seconds, or the encoded 'donE' completion message), written by firmware.
disp_en	1	in	Display-update enable/strobe, pulsed by firmware once per Timer tick.
seg_out[6:0]	7	out	Segment drive lines (a–g) to the physical seven-segment display.
digit_sel[3:0]	4	out	Digit-select / multiplex lines for a multi-digit display.

10. Watchdog Timer (Safety Interlock)

Signal	Width	Dir.	Description
wdt_trigger	1	in	Asserted by firmware (or derived directly from Timer irq_done) on dose completion, arming the lock.
wdt_stall_detect	1	internal	Independent stall/timeout detection: if the core fails to service the session within its expected window, the Watchdog latches the same locked state without firmware involvement.
wdt_locked	1	out	Latched lock-status flag; once set, remains set regardless of any subsequent bus or firmware activity.
pwm_force_disable	1	out	Hardware line into the PWM Generator that forces pwm_out low while wdt_locked is set (see PWM Generator, above).
alarm_out	1	out	Drives the alarm indicator (LED/buzzer) directly, overriding the GPIO-driven alarm path, for as long as the lock is latched.

wdt_reset_clear	1	in	Tied to system rst_n; this is the only way to clear wdt_locked and re-enable the PWM Generator.
-----------------	---	----	---

11. External / Off-Chip Signals

Signal	Width	Dir.	Description
Volume-entry switches / keypad	8	→ GPIO	Caregiver-entered liquid medication volume, in mL.
System reset button	1	→ core, → WDT	Manual reset; the only way to clear a latched Watchdog lock and start a new session.
Piezo Driver input	1	← PWM	Analog driver stage for the piezoelectric nebulizing element; receives pwm_out and is force-held low by the Watchdog after dose completion.
Seven-segment display (physical)	7 + digit sel	← 7-seg ctrl	Shows time-remaining during countdown and the 'donE' message at completion.
Alarm indicator (LED/buzzer)	1	← GPIO / ← WDT	Signals session completion / lockout state to the caregiver.

4. Control Path

The control path covers every signal that decides *when* and *whether* a value moves, rather than carrying a value itself: enables, strobes, chip-selects, interrupts, resets, and the hardware safety lock. NebuCore layers three independent control domains on top of a single shared data path, so a fault in one domain cannot corrupt another.

4.1 Bus-level control

Generic control that gates every transaction on the shared memory-mapped bus, regardless of which peripheral is addressed.

Signal	Source → Dest.	Role
clk, rst_n	System → all blocks	System clock and active-low reset common to core, bus, and every peripheral.
bus_we	Core → Bus	Write-enable; qualifies bus_wdata as a valid write.
bus_sel	Core → Bus	Byte/word select strobe for the current transaction.
bus_ack / s_ack	Peripheral → Bus → Core	Acknowledge/ready, used to extend a transaction where needed.
s_sel[N-1:0]	Bus → Peripherals	Per-peripheral chip-select decoded from the address.

4.2 Firmware-level control

Application-specific sequencing signals, all set and re-asserted by firmware running on the RISC-V core.

Signal	Block	Role
timer_en	Timer	Starts/stops the session countdown.
irq_timer_done	Timer → Core	Interrupt on each 1 s tick and again at dose-complete.
pwm_en	PWM Generator	Actuator enable; firmware must re-assert this once per tick, so a core stall lets it lapse.
disp_en	7-Seg Display Ctrl	Pulsed by firmware once per tick to refresh the displayed value.
gpio_dir[7:0]	GPIO	Per-pin direction configuration.
gpio_irq	GPIO → Core	Interrupt-on-change for volume-entry inputs, used instead of polling.
tx_valid / tx_ready	UART	Handshake: firmware loads a byte; peripheral flags when the transmit buffer is free.

4.3 Hardware safety control (independent of firmware)

The Watchdog Timer's outputs are the one control domain deliberately kept outside firmware's reach, so a firmware fault or race condition cannot leave the actuator enabled.

Signal	Source → Dest.	Role
wdt_trigger	Timer/Firmware → Watchdog	Arms the lock on dose completion.
wdt_stall_detect	Watchdog (internal)	Independently arms the lock if the core fails to service the session in time — no firmware involvement.
wdt_locked	Watchdog	Latched lock-status flag; once set, stays set regardless of later bus or firmware activity.

Signal	Source → Dest.	Role
pwm_force_disable	Watchdog → PWM Generator	Hardware override that forces pwm_out low while wdt_locked is set, independent of pwm_en.
alarm_out	Watchdog → Alarm indicator	Drives the alarm directly, overriding the GPIO-driven alarm path, for as long as the lock is latched.
wdt_reset_clear	rst_n → Watchdog	The only way to clear wdt_locked and re-enable the PWM Generator.

In short: bus-level control (4.1) is generic transaction plumbing, firmware-level control (4.2) is where the dosage-control application logic lives, and hardware safety control (4.3) sits outside both — it is the actual safety argument of the design.

5. Data Path

The data path covers every signal that carries an actual value being processed by the system — the entered volume, the computed duration, PWM duty cycle, display digits, instruction/data words, and log bytes — as opposed to the control path, which only decides when and whether those values move. NebuCore's data path is a single, mostly linear pipeline: volume → duration → duty cycle / countdown → PWM waveform and display digits.

5.1 Core ↔ Memory data path

Signal	Direction	Value carried
iaddr → idata	Core → Instr. Mem → Core	Instruction word fetched for the current program counter.
daddr ↔ dwdata / drdata	Core ↔ Data Mem	Entered volume, computed duration, and session state being loaded or stored.

5.2 Core ↔ Bus ↔ Peripheral data path

Register values moving between the core and each peripheral over the shared bus. The address lines steer this data but do not themselves carry a payload, so they are treated as part of the control path (see Section 4.1).

Signal	Direction	Value carried
bus_wdata / m_wdata / s_wdata	Core → Bus → Peripheral	Register writes: timer load value, PWM duty cycle, display digits, UART byte.
bus_rdata / m_rdata / s_rdata	Peripheral → Bus → Core	Register reads: live timer count, GPIO input value, UART received byte.

5.3 Timer data path

Signal	Direction	Value carried
timer_load[15:0]	Firmware → Timer	Treatment duration in seconds, from the mL-to-time conversion.
timer_count[15:0]	Timer → Firmware	Live countdown value, read once per tick to refresh the display.

5.4 GPIO data path

Signal	Direction	Value carried
gpio_in[7:0]	Switches/keypad → GPIO → Firmware	Raw caregiver-entered mL volume value.
gpio_out[7:0]	Firmware → GPIO → Off-chip	Display control bits and the pre-latch alarm output.

5.5 PWM Generator data path

Signal	Direction	Value carried
duty_cycle[7:0]	Firmware → PWM Gen	Duty-cycle setting controlling nebulizing intensity.

Signal	Direction	Value carried
pwm_out	PWM Gen → Piezo Driver	Resulting drive waveform sent to the external analog piezo driver.

5.6 Seven-Segment Display data path

Signal	Direction	Value carried
digit_data[15:0]	Firmware → Display Ctrl	Value to display: countdown seconds, or the encoded 'donE' message.
seg_out[6:0] / digit_sel[3:0]	Display Ctrl → Physical display	Segment pattern and digit-select lines actually driving the LEDs.

5.7 UART data path

Signal	Direction	Value carried
tx_data[7:0] → tx	Firmware → UART → Host	Log byte for session start, completion, or fault/alarm events.
rx → rx_data[7:0]	Host → UART → Firmware	Received byte (reserved for future host commands/diagnostics).

Note that the Watchdog Timer (Section 2.1) has no data path of its own — every one of its signals (wdt_trigger, wdt_locked, pwm_force_disable, alarm_out) enforces a state rather than carrying a value, so it belongs entirely to the control path in Section 4.3.

6. Common Errors / Failure Modes

This section catalogs realistic failure modes for NebuCore, grouped by where they originate in the design. These are the classes of bugs the RTL simulation testbench should be built to catch before integration.

6.1 Input / volume-entry errors

- **Volume value overflow / register mismatch** — if `gpio_in[7:0]` is 8 bits but the mL-to-time lookup only covers a smaller range, an out-of-range entry can truncate or wrap silently instead of being rejected.
- **Zero-volume entry** — an entry of 0 mL sets `timer_load` to 0; the countdown interrupt fires immediately and the device can report 'donE' without ever nebulizing anything.
- **No input validation range check** — firmware must clamp/reject values outside the pre-characterized lookup range before writing `timer_load[15:0]`, otherwise it interpolates or extrapolates into an uncharacterized duration.
- **Debounce/glitch on switches** — `gpio_in` sampled mid-transition on mechanical switches can corrupt the volume value if there is no debounce logic in the GPIO input path.
- **Re-entry during an active session** — changing the volume switches while the timer is running can recompute `timer_load` mid-session and desync the display from the dose actually being delivered.

6.2 Volume-to-time conversion errors

- **Lookup table boundary/rounding errors** — off-by-one at table edges, or truncation instead of proper rounding when interpolating between characterized points.
- **Fixed-point arithmetic overflow** — insufficient headroom in the mL→seconds multiplication can overflow the intermediate result before it is stored in `timer_load[15:0]`.
- **Unit mismatch** — a single wrong scaling factor (e.g. mL×10 vs mL×100) silently produces a plausible-looking but wrong duration — dangerous because it does not crash, it under- or over-doses.

6.3 Timing / synchronization errors

- **pwm_en refresh race** — if the once-per-tick refresh write and the Timer's decrement are not properly synchronized, a one-cycle gap can glitch `pwm_en` low, causing an unintended actuator flicker.
- **Interrupt latency vs. tick period** — if `irq_timer_done` is not serviced before the next tick, ticks can be missed or double-counted, shortening or lengthening the actual dose time relative to the display.
- **Clock domain crossing** — `tick_1s` is generated internally to the Timer; without proper synchronization to the bus clock domain, metastability can corrupt `timer_count` reads intermittently.
- **Display update skew** — if `disp_en` and the `pwm_en` refresh are not tied to the same tick edge, the displayed countdown and the actual actuation time can drift apart over a long session.

6.4 Bus / register interface errors

- **Address decode collision** — two peripherals responding to overlapping `s_sel` ranges due to a memory-map bug causes `bus_rdata` to be muxed incorrectly — a classic bus-fabric bug, often missed until integration testing.
- **Unacknowledged transaction / bus hang** — if a peripheral never asserts `bus_ack` (e.g. a write to the wrong register offset), the core can stall indefinitely waiting for a slave that will not respond, absent a bus timeout.
- **Read-modify-write hazards** — if `pwm_en` and `duty_cycle` share a register, a race between the tick-refresh write and any other firmware write can unintentionally clear the enable bit.

6.5 Watchdog / safety-interlock corner cases

- **wdt_trigger never asserted** — a firmware bug that skips this write leaves the actuator disabled only by pwm_en lapsing, while the hardware lock and alarm_out never latch — a silent failure with no lockout indication.
- **Stall-detection threshold too loose/tight** — wdt_stall_detect's timeout window needs careful characterization: too short causes false lockouts on normal firmware delays; too long lets a real stall leave the actuator running longer than intended.
- **Reset-clear race** — since wdt_reset_clear is tied to rst_n, a reset pulse that is too short (a glitch) may not hold long enough to fully clear wdt_locked.
- **Watchdog false-triggers from legitimate long operations** — a slow firmware task at completion time (e.g. a UART log flush) can look like a stall to wdt_stall_detect and cause a spurious lock even though the dose completed correctly.

6.6 Display / UART errors

- **Display digit-multiplexing timing bug** — if digit_sel[3:0] cycles too slowly, digits can appear dim or flicker; if too fast, timing margins should still be checked.
- **'donE' encoding collision** — if the encoded completion message shares bit patterns with valid countdown values, a corrupted digit_data write could display 'donE' prematurely or show garbage numerals mistaken for a real countdown.
- **UART overrun** — if tx_valid is asserted faster than tx_ready allows (e.g. a burst of events at session start plus an immediate fault), bytes can be dropped from the session log without an overrun flag being checked.

6.7 Verification-specific gaps

Test cases the RTL simulation testbench should explicitly cover:

- **Boundary volumes** — 0 mL, maximum mL, and one-past-maximum mL.
- **Missed pwm_en refresh** — firmware skips refreshing pwm_en for one tick — does the actuator correctly stay or go low?
- **Core stall with no firmware cooperation** — does the Watchdog independently latch the lock? This is the single most important test case for the entire safety argument of the project.
- **Reset during a latched lock** — reset asserted while wdt_locked is set — does it clear cleanly?
- **Simultaneous bus access** — concurrent access attempts to different peripherals in the same cycle — does arbitration/chip-select resolve correctly?

7. Interrupts

NebuCore uses a deliberately sparse interrupt structure — most of the design works by polling once per Timer tick rather than interrupting on every event. This section lists every interrupt source explicit in the signal list, notes their behavior, and flags gaps a full implementation would likely need to close.

7.1 Interrupts in the signal list

Interrupt	Source → Dest.	Trigger	Meaning
irq_timer_done	Timer → Core	Every 1 s tick, and again when timer_count reaches zero	Two-purpose: refresh display / re-assert pwm_en on each tick; signal dose-complete on the final tick.
irq_wdt	Watchdog → Core	Optional status interrupt, e.g. on wdt_stall_detect	Informational/logging only — the actual lock is enforced in hardware independent of this line.
gpio_irq	GPIO → Core	Interrupt-on-change on volume-entry switch/keypad inputs	Lets firmware react to a new volume entry without continuously polling gpio_in.

7.2 Interrupt behavior worth noting

- **irq_timer_done is overloaded** — since it fires on every tick and on completion, firmware must read timer_count inside the handler to distinguish 'just a tick' from 'dose done' — a common source of the timing bugs in Section 6.3.
- **irq_wdt is deliberately non-critical** — the interrupt path can be missed, delayed, or disabled and the safety lock still engages, because wdt_stall_detect and pwm_force_disable are hardwired, not interrupt-driven. If irq_wdt were the only enforcement mechanism, it would defeat the purpose of the hardware-independent watchdog.
- **gpio_irq is optional by design** — framed as 'used instead of polling' — the reference architecture works with polling alone, so this interrupt is an efficiency feature, not a functional requirement.

7.3 Interrupts not yet in the design (common gaps)

Not part of the design as specified, but worth flagging as common gaps in a first-pass interrupt scheme:

- **UART RX interrupt** — rx_valid is currently just a status flag. If the host needs to send commands (rx is reserved for future host commands), that path will need its own interrupt or the core must poll it.
- **Bus timeout / fault interrupt** — nothing currently interrupts the core if bus_ack never arrives (see the bus-hang failure mode in Section 6.4). A bus-watchdog interrupt would let firmware recover instead of stalling indefinitely.
- **UART TX-ready interrupt** — tx_ready is presumably polled before writing tx_data; under load (e.g. logging a fault burst) this could need to become a real interrupt source if polling isn't fast enough.

7.4 Priority consideration

If multiple interrupts can be pending simultaneously, irq_timer_done (dose-complete case) should take priority over gpio_irq — a new volume entry mid-session must not be able to delay the core from servicing dose completion and handing off to the Watchdog.